

Calculate Cyclic Redundancy Check (CRC) based on binary string Data and generator polynomial

- CRC method used for check and aware of occurred error in data link layer while transmitting data
- That works with binary XOR division operator
- The remainder of above operation tell us if error occurs during transmitting data

```
print_result_bool = True
```

Define XOR operator function used for binary division in calculate CRC

```
def xor(a, b):  
    if a == b:  
        return "0"  
    else:  
        return "1"
```

Binary XOR division used for division r-bits size of binary data by generator polynomial in binary string form

```
def binary_xor_division(dividend, divisor):  
  
    dividend = list(dividend)  
    divisor_length = len(divisor)  
  
    for i in range(len(dividend) - divisor_length + 1):  
        if dividend[i] == "1":  
            for j in range(divisor_length):  
                dividend[i + j] = xor(dividend[i + j], divisor[j])  
  
    remainder_length = divisor_length - 1  
    remainder = "".join(dividend[-remainder_length:])  
  
    return remainder  
  
def calculate_crc(data, generator):  
    r = len(generator) - 1  
    zero_filled_data = data + ("0" * r)  
    crc = binary_xor_division(zero_filled_data, generator)
```

```
    return crc, zero_filled_data
```

Create codeword by adding binary data to CRC code at the end

```
def create_codeword(data, crc):  
    return data + crc
```

Based on generator polynomial checks if inputted codeword has error or not using codeword binary XOR division by generator polynomial

- After division if remainder was zero it means codeword is correct and not involved with error else has involved

```
def check_codeword(codeword, generator):  
  
    remainder = binary_xor_division(codeword, generator)  
  
    if set(remainder) == {"0"}:  
        return True, remainder  
    else:  
        return False, remainder
```

Initialize binary string data and generator polynomial with checking

```
# enter a binary string as input has less than 33 bits  
data = "1101011011"  
  
# enter the binary string representing your generator polynomial  
generator = "10011"  
  
# Some basic checks on inputted binary string  
if not all(bit in "01" for bit in data):  
    print("Binary string should only contain 0 and 1.")  
    print_result_bool = False  
  
if not all(bit in "01" for bit in generator):  
    print("Binary string should only contain 0 and 1.")  
    print_result_bool = False  
  
if len(data) > 32:  
    print("Binary string input must be equal or less than 32-bits.")  
    print_result_bool = False
```

```

if len(generator) < 2 or generator[0] != "1" or generator.count("1") == 0:
    print("Entered binary string does not representing any valid generator polynomial.")
    print_result_bool = False

```

Results based on user data input and generator polynomial

```

if print_result_bool:
    # prints degree of generator polynomial
    r = len(generator) - 1
    print("\n Degree of generator polynomial is:", r)

    crc, zero_filled_data = calculate_crc(data, generator)
    print("Value of CRC binary code is:", crc)

    codeword = create_codeword(data, crc)
    print("Codeword is:", codeword)

    is_valid, remainder = check_codeword(codeword, generator)
    print("Remainder after XOR division is:", remainder)

    if is_valid:
        print("Codeword is correct and not involved with error.")
    else:
        print("Codeword is not correct and involved with error.")

Degree of generator polynomial is: 4
Value of CRC binary code is: 1110
Codeword is: 1101011011110
Remainder after XOR division is: 0000
Codeword is correct and not involved with error.

```

Enter an input codeword as binary string to check if correct or not

```

received_codeword = "101110111100"

if not all(bit in "01" for bit in received_codeword):
    print("Binary string should only contain 0 and 1.")
    print_result_bool = False

if print_result_bool:
    is_valid, remainder = check_codeword(received_codeword, generator)

    print("Remainder after XOR division for inputted codeword is:", remainder)

```

```
if is_valid:
    print("Inputted codeword is correct and not involved with error.")
else:
    print("Inputted codeword is not correct and involved with error.")
```

Remainder after XOR division for inputted codeword is: 0011
Inputted codeword is not correct and involved with error.